

Worksheet — 2.3 Algorithms

Name: ___ Date: _____

OCR A Level Computer Science H446, Component 02, Section 2.3.1 — Algorithms.

Time: ~40 min. SHOW YOUR TRACES.

Activity 1 — Big O match

Match each algorithm to its **best**, **average** and **worst** time complexity by writing the correct Big O in each cell. Choose from: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$.

Algorithm	Best	Average	Worst
Linear search			
Binary search			
Bubble sort (with swap flag)			
Insertion sort			
Merge sort			
Quick sort			

Activity 2 — Trace a sort (bubble OR insertion)

Sort this list into **ascending** order: [5, 1, 4, 2, 8]

Circle which you are tracing: **BUBBLE / INSERTION**

Fill in the list state **after each pass**. Add rows if you need them.

Pass	List state after this pass	Swaps this pass
Start	5 1 4 2 8	—
1	— — — — —	
2	— — — — —	
3	— — — — —	
4	— — — — —	

Final sorted list: _ _ _ _ _

Activity 3 — Trace a binary search

Sorted list (index 0–8):

Index	0	1	2	3	4	5	6	7	8
Value	3	7	11	15	19	23	27	31	35

Search for 27. Use `mid = (low + high) DIV 2` (integer division, round down). Fill the table.

Step	low	high	mid (index)	value at mid	< = > target?
1					
2					
3					
4					

Found at index: Number of comparisons:

Activity 4 — Big O justification (short answer)

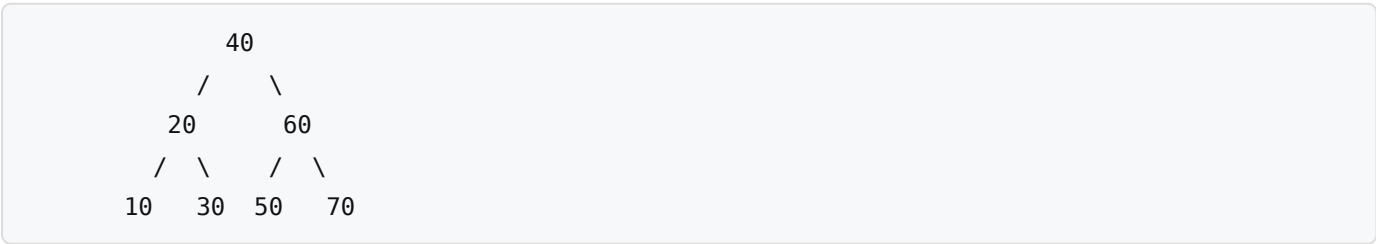
(a) Explain why binary search is **$O(\log n)$** . Refer to what happens to the list each comparison.

(b) Explain why bubble sort is **$O(n^2)$** in the average case.

(c) State the precondition binary search requires that linear search does not.

Activity 5 — Tree traversals

Binary tree (text form):



Write the output of each traversal:

Pre-order (Node → Left → Right):

In-order (Left → Node → Right):

Post-order (Left → Right → Node):

Which traversal outputs a BST in ascending order? __

Activity 6 — Dijkstra's shortest path

Weighted, undirected graph given as an **edge list** (node1 — node2 : weight):

A – B : 4
A – C : 2
B – C : 1
B – D : 5
C – D : 8
C – E : 10
D – E : 2
D – F : 6
E – F : 3

Find the shortest path and total distance from A to F.

Working table — write the tentative distance to each node, updating as you settle nodes. Cross out / overwrite when a shorter route is found.

Settled node (order)	A	B	C	D	E	F
Initial	0	∞	∞	∞	∞	∞

Shortest path A → F: → → → →

Total distance: _____

Challenge

Challenge box. A* search uses $f(n) = g(n) + h(n)$.

(a) For the graph above, suppose you are given a straight-line heuristic estimate to F of: $h(A)=9$, $h(B)=7$, $h(C)=8$, $h(D)=4$, $h(E)=3$, $h(F)=0$. Is this heuristic **admissible**? Explain how you would check, and what admissible means.

(b) State ONE situation where A explores fewer nodes than Dijkstra, and ONE condition under which A behaves exactly like Dijkstra.

...

...
